My Playlist 구현 시나리오

Nov 3, 2026 · @조림

현재 상태와 남은 일

서버의 플레이리스트·곡·좋아요·통계·YouTube 검색 API 23개는 구현되어 있고, 남은 일은 회원 API와 클라이언트입니다. 이 문서는 조원이 받은 코드를 실행하고, 같은 규칙으로 남은 부분을 구현하기 위한 안내입니다.

영역	상태	비고
플레이리스트 API	구현 완료	생성, 검색, 상세, 내 목록, 수정, 삭제
곡 관리 API	구현 완료	추가, 목록, 수정, 삭제, 순서 변경
좋아요 API	구현 완료	등록, 취소, 여부, 개수, 좋아요한 목록
통계 API	구현 완료	전체 통계 5개, 내 활동 통계 1개
YouTube 검색 API	구현 완료	API 키는 서버에만 보관
DB 구조	구현 완료	테이블 5개, 마이그레이션과 `schema.sql` 제공
회원 API	미구현	지금은 개발용 임시 인증 사용
클라이언트 UI	미구현	클라이언트 종류 결정 필요

기술 스택은 ASP.NET Core 8 Web API, EF Core, MySQL 8.0, Swagger입니다. 구현된 API는 임시 MySQL에서 자동 검사를 통과했습니다. 다만 실제 로그인과 클라이언트를 연결한 검사는 아직 하지 않았습니다.

개발 환경 준비

아래 6단계를 마치면 각자 컴퓨터에서 서버가 실행됩니다. 비밀번호와 API 키는 사람마다 다르므로 `appsettings.json`이 아니라 user-secrets에 넣습니다.

미리 설치할 것: .NET 8 SDK, MySQL 8.0, Visual Studio 또는 터미널.

1. EF 도구를 설치합니다. 이미 있으면 건너뜁니다.

```
dotnet tool install --global dotnet-ef --version 8.0.22
```

2. 저장소를 받고 그 폴더로 들어갑니다.
3. MySQL 연결 정보를 넣습니다.

```
dotnet user-secrets set "ConnectionStrings:DefaultConnection"
"Server=localhost;Port=3306;Database=my_playlist;User=root;Password=본인_비
밀번호;" --project MyPlaylist.Server
```

4. 본인의 YouTube Data API v3 키를 넣습니다. 키를 채팅이나 파일로 공유하지 마세요.

```
dotnet user-secrets set "YouTube:ApiKey" "본인_키" --project
MyPlaylist.Server
```

5. DB를 만들고 테스트 회원을 넣습니다.

```
dotnet ef database update --project MyPlaylist.Server
```

```
USE my_playlist;
INSERT INTO USER (user_name, user_pw, created_at)
VALUES ('alice', 'x', NOW()), ('bob', 'x', NOW());
```

6. 서버를 실행하고 http://localhost:5080/swagger 를 엽니다.

```
dotnet run --project MyPlaylist.Server
```

Swagger의 Authorize에 회원 번호 `1`을 넣으면 처음 넣은 회원으로 로그인한 것처럼 테스트할 수 있습니다. 이 개발용 인증(`X-Dev-User-Id` 헤더)은 Development 환경에서만 동작합니다.

DB는 각자의 컴퓨터에 따로 있어, 다른 조원이 만든 데이터는 보이지 않습니다.

회원 API 구현 시나리오

로그인 후 `ClaimTypes.NameIdentifier` 클레임에 `user_id`를 담으면, 이미 만든 API는 고치지 않고 연결됩니다. 회원 API는 `MyPlaylist.Server` 프로젝트에 추가하고, 이미 있는 `Models/User.cs`와 `AppDbContext`를 그대로 씁니다.

기능	Method, URL	구현 시나리오	주요 응답
회원가입	POST `/api/users/signup`	아이디 중복을 확인하고 비밀번호를 해시해 저장	201, 중복이면 409
로그인	POST `/api/users/login`	해시를 비교한 뒤 클레임에 `user_id`를 담아 인증 정보 발급	200, 실패하면 401
로그아웃	주소 미정	명세서 M-03. 현재 회원 API 목록에 없으므로 추가 필요	204
회원정보 조회	GET `/api/users/{userId}`	본인인지 확인하고 비밀번호를 뺀 정보 반환	200, 타인이면 403

기능	Method, URL	구현 시나리오	주요 응답
회원정보 수정	PUT `/api/users/{userId}`	본인 확인 후 수정. 아이디를 바꾸면 중복도 확인	204, 타인이면 403
회원탈퇴	DELETE `/api/users/{userId}`	본인 확인 후 `USER` 행만 삭제	204, 타인이면 403
아이디 중복 확인	GET `/api/users/check-name`	사용 가능 여부 반환	200

응답 코드는 기존 API에 맞춘 제안이며, 담당 조원이 확정합니다.

구현할 때 지킬 규칙입니다.

- **비밀번호:** `user_pw`에는 해시만 저장하고 어떤 응답에도 넣지 않습니다.
- **본인 확인:** URL의 `userId`를 믿지 말고 `User.GetUserId()`와 비교합니다. 같지 않으면 403입니다.
- **아이디 중복:** 사전 확인과 함께 DB의 중복 오류도 409로 처리합니다. `PlaylistLikesController`의 처리 방식과 같습니다.
- **주소:** `{userId:int}`로 제한합니다. `check-name`과 `me`가 회원 번호로 해석되지 않게 하기 위해서입니다.
- **탈퇴:** 관련 데이터를 따로 지우지 않습니다. 회원의 플레이리스트와 곡, 좋아요는 DB가 함께 지우고, 다른 플레이리스트의 조회 기록은 `user_id`만 NULL이 됩니다.
- **탈퇴 후 남은 인증 정보:** `DeletedUserMiddleware`가 비로그인으로 처리합니다.

실제 로그인을 넣을 때는 `Program.cs`의 "인증 등록" 부분만 교체합니다. 다음은 명세서의 세션 방식에 맞춘 쿠키 인증 예시이며, 아직 실행해 보지는 않았습니다.

```
// Program.cs: 쿠키 인증 등록. API는 로그인 페이지 이동 대신 401을 반환한다.
builder.Services
.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
.AddCookie(options =>
{
    options.Events.OnRedirectToLogin = context =>
    {
        context.Response.StatusCode = 401;
        return Task.CompletedTask;
    };
});

// 로그인 성공 후: NameIdentifier에 user_id를 담는 것이 핵심이다.
var identity = new ClaimsIdentity(
new[] { new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()) },
CookieAuthenticationDefaults.AuthenticationScheme);
```

```
await HttpContext.SignInAsync(new ClaimsPrincipal(identity));

// 로그아웃
await HttpContext.SignOutAsync();
```

교체한 뒤에는 Swagger의 Authorize 대신 로그인 API를 호출해 테스트합니다. Swagger 설정의 개발용 헤더 부분과 `Auth/DevAuthHandler.cs`는 그때 지워도 됩니다.

클라이언트 구현 시나리오

클라이언트는 아래 흐름 순서대로 만들면, 회원 API가 없어도 개발용 헤더로 먼저 개발할 수 있습니다. 요청 헤더를 붙이는 곳을 한 곳으로 모아 두면 실제 로그인으로 쉽게 바꿀 수 있습니다.

1. 비회원이 공개 플레이리스트를 찾는다

1. 메인 화면에서 인기 목록을 `GET /api/statistics/playlists/popular?sort=likes&limit=10`로 불러옵니다.
2. 검색창에 입력하면 `GET /api/playlists?keyword=검색어&sort=latest&page=1&size=20`을 호출합니다. 제목과 아이디를 함께 검색합니다.
3. 정렬 선택은 `sort` 값으로 보냅니다. 최신순은 `latest`, 좋아요순은 `likes`, 클릭순은 `views`입니다.
4. 응답의 `totalCount`로 페이지 수를 계산합니다.

2. 플레이리스트 상세를 본다

1. 목록에서 항목을 누르면 `GET /api/playlists/{playlistId}`를 **한 번만** 호출합니다. 호출할 때마다 클릭 수가 1 오르므로, 좋아요를 누른 뒤 화면을 갱신할 때는 다시 호출하지 않습니다.
2. 응답의 `playlist`, `likedByMe`, `tracks`로 화면을 그립니다.
3. 곡을 누르면 `youtubeUrl`로 이동합니다. 썸네일은 `thumbnailUrl`을 씁니다.
4. 비공개 플레이리스트를 타인이 열면 403이 오므로 안내 문구를 보여 줍니다.

3. 회원이 플레이리스트를 만들고 곡을 넣는다

1. 제목, 설명, 공개 여부를 입력해 `POST /api/playlists`로 생성합니다. `isPublic`을 빼면 비공개로 만들어지니 값을 항상 보냅니다.

   ```
   { "playlistTitle": "새벽 감성 노래", "description": "잠 안 올 때", "isPublic": true }
   ```

2. 검색 버튼을 누를 때 `GET /api/youtube/search?keyword=검색어&maxResults=10`을 호출합니다. 검색 1회에 일일 할당량 100을 쓰므로 입력할 때마다 자동 검색하지 않습니다.

3. 결과에서 곡을 고르면 `POST /api/playlists/{playlistId}/tracks`로 추가합니다. `artist`에는 채널명이 들어오므로, 저장 전에 고칠 수 있게 합니다.

```
{ "youtubeVideoId": "BzYnNdJhZQw", "musicTitle": "밤편지", "artist": "아이유"
}
```

4. `youtubeUrl`과 `sortOrder`는 서버가 정하므로 보내지 않습니다. 같은 곡을 두 번 넣을 수 있고, 곡들은 `trackId`로 구분합니다.

4. 곡과 플레이리스트를 관리한다

1. 순서를 바꾸면 전체 `trackId`를 바뀐 순서대로 `PATCH /api/playlists/{playlistId}/tracks/order`에 보냅니다. 일부만 보내면 400입니다.

```
{ "trackIds": [3, 1, 2] }
```

2. 곡 삭제는 `DELETE /api/playlists/{playlistId}/tracks/{trackId}`입니다.
3. 수정은 `PUT /api/playlists/{playlistId}`입니다. 보내지 않은 값은 비워지므로 제목, 설명, 공개 여부를 모두 보냅니다.
4. 삭제는 `DELETE /api/playlists/{playlistId}`입니다. 곡, 좋아요, 클릭 기록이 함께 지워지므로 확인 창을 띄웁니다.
5. 수정·삭제 버튼은 본인 플레이리스트에만 보여 줍니다. 서버도 타인의 요청을 403으로 막습니다.

5. 좋아요를 누른다

1. 상세 응답의 `likedByMe`로 버튼 상태를 정합니다.
2. 등록은 `POST`, 취소는 `DELETE /api/playlists/{playlistId}/likes`입니다. 본인 플레이리스트에도 누를 수 있습니다.
3. 성공하면 `GET /api/playlists/{playlistId}/likes/count`로 개수만 갱신합니다.
4. 409는 이미 누른 상태라는 뜻이므로 오류 창 대신 버튼을 누른 상태로 표시합니다.

6. 마이페이지를 본다

화면 영역	API	비고
내 플레이리스트	`GET /api/users/me/playlists`	비공개 포함
좋아요한 목록	`GET /api/users/me/likes`	타인이 비공개로 바꾼 항목은 제외
내 활동 통계	`GET /api/statistics/users/me`	만든 목록, 곡, 누른·받은 좋아요, 받은 클릭 수

오류 처리 공통 규칙

- **401:** 로그인 화면으로 보냅니다. 검색, 추가, 좋아요 중에도 나올 수 있습니다.
- **400, 403, 404, 409:** 응답 본문의 한글 메시지를 보여 줍니다. 단, 입력값 검증의 400은 JSON 형식으로 옵니다.
- **503:** 검색 화면에 "검색을 사용할 수 없음"을 표시합니다. 이미 만든 플레이리스트는 계속 쓸 수 있습니다.

완료 확인 시나리오

회원 API와 클라이언트를 연결한 뒤 아래 항목을 차례로 확인하면 주요 흐름을 모두 지나갑니다. 회원 A와 회원 B, 두 계정으로 진행합니다.

- [ ] A, B가 회원가입하고 같은 아이디로 다시 가입하면 409가 나온다
- [ ] A가 로그인해 공개 플레이리스트 1개와 비공개 플레이리스트 1개를 만든다
- [ ] A가 YouTube에서 곡을 검색해 공개 플레이리스트에 3곡을 넣는다
- [ ] A가 곡 순서를 바꾸고 다시 불러와도 순서가 유지된다
- [ ] 곡을 누르면 유튜브 링크로 이동한다
- [ ] 로그아웃한 상태에서 공개 플레이리스트는 검색되고 비공개는 보이지 않는다
- [ ] 로그아웃한 상태에서 상세를 열어도 클릭 수가 1 오른다
- [ ] B가 A의 공개 플레이리스트에 좋아요를 누르고, 다시 등록하면 409가 나온다
- [ ] B가 A의 플레이리스트를 수정하거나 삭제하면 403이 나온다
- [ ] B가 A의 회원정보 URL로 조회·수정·탈퇴하면 403이 나온다
- [ ] A의 마이페이지 통계에 곡 수, 받은 좋아요, 클릭 수가 맞게 나온다
- [ ] B의 좋아요한 목록에 A의 플레이리스트가 보이고, A가 비공개로 바꾸면 사라진다
- [ ] B가 탈퇴하면 A의 좋아요 수는 줄고 클릭 수는 유지된다
- [ ] 탈퇴한 B의 인증 정보로 플레이리스트를 만들면 401이 나온다
- [ ] A가 플레이리스트를 삭제하면 곡, 좋아요, 클릭 기록이 함께 지워진다

상태 코드

코드	의미	클라이언트 처리
200, 201, 204	성공	204는 본문이 없으므로 읽지 않음
400	입력값 오류	입력 항목 안내
401	로그인 필요	로그인 화면으로 이동
403	본인 소유가 아니거나 비공개	권한 안내

코드	의미	클라이언트 처리
404	대상 없음	목록으로 돌아감
409	중복 좋아요, 중복 아이디	상황에 맞게 표시
502, 504	YouTube 호출 실패, 시간 초과	다시 시도 안내
503	키 미설정, 할당량 초과 또는 키 제한	검색 불가 안내

팀 결정 사항과 협업 규칙

구현에 들어가기 전에 아래 5가지를 정해야 합니다. 특히 로그인 방식과 클라이언트 종류는 다른 작업에 영향을 줍니다.

결정할 것	현재 상황	영향
로그인 방식 (쿠키 세션, JWT)	명세서에는 "세션 종료"로 되어 있음	회원 API 구현, 클라이언트의 요청 방식
클라이언트 종류	명세서에는 웹 앱으로 되어 있음	서버와 다른 주소에서 웹을 띄우면 CORS 설정 필요
로그아웃 API	명세서에만 있고 회원 API 목록에는 없음	주소 확정과 목록 추가 필요
회원탈퇴, 정보 조회·수정, 아이디 확인	회원 API 목록에만 있고 명세서에는 없음	명세서에 추가 필요
통계 API	구현되어 있지만 명세서에는 영역이 없음	명세서에 추가 필요

코드를 함께 고칠 때는 다음을 지킵니다.

- **비밀번호와 키:** 코드, `appsettings.json`, 채팅에 넣지 않고 user-secrets만 씁니다.
- **브랜치:** 자기 브랜치에서 작업하고, 합치기 전에 완료 확인 시나리오 중 관련 항목을 확인합니다.
- **마이그레이션:** DB 구조를 바꾸기 전에 팀에 알리고, 만든 파일을 함께 올립니다. 받은 사람은 `dotnet ef database update`를 다시 실행합니다.
- **API 변경:** URL이나 응답 필드를 바꾸면 `README.md`와 이 문서를 함께 고칩니다.
- **YouTube 할당량:** 각자 본인의 키를 씁니다. 기본 일일 할당량 10,000이면 하루 약 100번 검색할 수 있습니다.

아직 정해지지 않은 것: 저장소 주소와 화면별 담당자.